LeetCode 383: Ransom Note

1. Problem Title & Link

Title: LeetCode 383: Ransom Note

Link: https://leetcode.com/problems/ransom-note/

2. Problem Statement (Short Summary)

Given two strings:

ransomNote

magazine

Return:

true

if the ransom note can be constructed using letters from the magazine.

Each character in magazine can be used **only once**.

Otherwise return:

false

Example

Input:

ransomNote = "aa"

magazine = "aab"

Output:

true

Because:

magazine contains

a → 2 times

b → 1 time

Ransom note needs:

a → 2 times

Enough characters exist.

3. Examples (Input → Output)

Example 1

Input:

ransomNote = "a"

magazine = "b"

Output:

false

Example 2

Input:

ransomNote = "aa"

magazine = "ab"

Output:

false

Need:

2 a's

Have:

1 a

Example 3

Input:

ransomNote = "aa"

magazine = "aab"

Output:

true

Example 4

Input:

ransomNote = "abc"

magazine = "cbad"

Output:

true

4. Constraints

1 <= ransomNote.length, magazine.length <= 10^5

Both contain lowercase English letters.

Important:

Each magazine character can be used only once.

5. Core Concept (Pattern / Topic)

HashMap / Frequency Counting

Secondary Topics:

- Character Counting
- Array Frequency

6. Thought Process (Step-by-Step Explanation)

Understanding the Problem

Suppose:

ransomNote = "aa"

magazine = "ab"

Need:

a → 2

Available:

a → 1

b → 1

Not enough.

Return:

false

Brute Force Idea

For every character in ransom note:

Search magazine.

Mark used characters.

Problem

Complexity becomes:

$O(n^2)$

Too slow.

Better Observation

We only care about:

Character Frequencies

Example:

magazine = "aab"

Count:

a → 2

b → 1

Now process ransom note:

aa

Use:

a → 1 left

a → 0 left

Success.

Key Insight

Store available characters.

Then consume them while reading ransom note.

If any count becomes negative:

Not enough letters

Return:

false

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
ransomNote = aa

magazine = aab
Frequency:
a → 2
b → 1
Use first:
a

a → 1
Use second:
a

a → 0
Success.
Return:
true
```

8. Pseudocode (Language Independent)

```
count frequencies of magazine

for each char in ransomNote

    decrease frequency

    if frequency < 0

         return false

return true
```

9. Code Implementation

Python

```python
class Solution:
    def canConstruct(
        self,
        ransomNote: str,
        magazine: str
    ) -> bool:

        count = [0] * 26

        for ch in magazine:
            count[ord(ch) - ord('a')] += 1

        for ch in ransomNote:

            idx = ord(ch) - ord('a')

            count[idx] -= 1

            if count[idx] < 0:
                return False

        return True
```

Java

```java
class Solution {

    public boolean canConstruct(
            String ransomNote,
            String magazine) {
```

```
        int[] count = new int[26];

        for(char ch :
                magazine.toCharArray()) {

            count[ch - 'a']++;
        }

        for(char ch :
                ransomNote.toCharArray()) {

            count[ch - 'a']--;

            if(count[ch - 'a'] < 0)
                return false;
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n + m)

Where:

n = ransomNote length

m = magazine length

Space Complexity

O(1)

Frequency array size is always:

26

11. Common Mistakes / Edge Cases

Mistake 1

Using Set instead of Frequency Count.

Example:

aa

ab

Sets:

{a}

{a,b}

Appear valid.

But answer is:

false

Need frequency.

Mistake 2

Checking only character existence.

Need enough occurrences too.

Mistake 3

Not stopping when count becomes negative.

Edge Cases

Empty Requirement (hypothetical)

Would return:

true

Nothing to construct.

Exact Match

Input:

abc

abc

Output:

true

Missing Character

Input:

abc

ab

Output:

false

12. Detailed Dry Run

Input:

ransomNote = "aa"

magazine = "aab"

Frequency after magazine:

Character	Count
a	2
b	1

Process ransom note:

Character	New Count
a	1
a	0

Never negative.

Return:

true

13. Common Use Cases (Real-Life / Interview)

Resource Allocation

Checking whether enough resources exist.

Inventory Management

Can requested items be fulfilled?

Character Availability Problems

Common interview frequency-count pattern.

Interview Purpose

Tests:

HashMap

Counting

Frequency Arrays

14. Common Traps (Important!)

Trap 1

Using Set.

Need frequency counts.

Trap 2

Comparing only lengths.

Example:

ransomNote = aa

magazine = bb

Same length.

Still impossible.

Trap 3

Not recognizing similarity to:

Valid Anagram

Both are frequency-count problems.

15. Builds To (Related LeetCode Problems)

LC 383 - Ransom Note

↓

LC 242 - Valid Anagram

↓

LC 49 - Group Anagrams

↓

LC 438 - Find All Anagrams

↓

LC 567 - Permutation in String

These are core frequency-counting problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Nested Loops	$O(n^2)$	O(1)	Poor
HashMap	O(n+m)	O(1)/ O(26)	Good
Frequency Array	O(n+m)	O(1)	Best

Approach 1: Nested Search

Search every character manually.

Too slow.

Approach 2: HashMap

Works for all character sets.

Approach 3: Frequency Array (Chosen)

Since only lowercase letters exist.

Most efficient.

17. Why This Solution Works (Short Intuition)

The magazine provides a supply of characters.

The ransom note consumes those characters.

If at any point a required character exceeds the available supply, construction becomes impossible.

18. Variations / Follow-Up Questions

Follow-Up 1

Support uppercase and lowercase characters.

Use:

HashMap

instead of a 26-size array.

Follow-Up 2

Return the missing characters.

Example:

Need: aaab

Have: aab

Missing:

a

Follow-Up 3

Find whether two strings are anagrams.

Related:

LC 242

Follow-Up 4

Find all anagrams in a string.

Related:

LC 438

Interview Takeaway

The key pattern is:

Frequency Counting

Whenever you hear:

Enough characters?

Available resources?

Character inventory?

think:

HashMap

Frequency Array

This is one of the most common patterns in easy and medium string interview questions.